

Source code for phase 3

📦 Import libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
roc_curve, auc
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import files
```

📁 Upload CSV file
uploaded = files.upload()

Load the dataset
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

🇸🇬 Quick overview of the data
print("\nFirst 5 rows of the data:")
print(df.head())
print("\nData Info:")
print(df.info())

🛠️ Handle missing values (if any)
df.fillna(df.mean(), inplace=True)

🪄 Encode categorical features if needed
for column in df.select_dtypes(include=['object']).columns:
 le = LabelEncoder()
 df[column] = le.fit_transform(df[column])

🦸 Feature scaling
X = df.drop('Class', axis=1) # 'Class' is the target variable (fraud: 1, not fraud: 0)
y = df['Class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

🦄 Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

🚀 Train Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

```

# 🚀 Train Logistic Regression model
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)

# 📊 Evaluate the models
print("\nRandom Forest Evaluation:")
print(classification_report(y_test, y_pred_rf))
print("Accuracy (Random Forest):", accuracy_score(y_test, y_pred_rf))

print("\nLogistic Regression Evaluation:")
print(classification_report(y_test, y_pred_log))
print("Accuracy (Logistic Regression):", accuracy_score(y_test, y_pred_log))

# 📊 Plot confusion matrices for both models
fig, ax = plt.subplots(1, 2, figsize=(14, 6))

# Confusion Matrix for Random Forest
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues', ax=ax[0])
ax[0].set_title("Random Forest Confusion Matrix")
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

# Confusion Matrix for Logistic Regression
conf_matrix_log = confusion_matrix(y_test, y_pred_log)
sns.heatmap(conf_matrix_log, annot=True, fmt='d', cmap='Blues', ax=ax[1])
ax[1].set_title("Logistic Regression Confusion Matrix")
ax[1].set_xlabel('Predicted')
ax[1].set_ylabel('Actual')

plt.show()

# 📈 ROC Curve for both models
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_model.predict_proba(X_test)[:,1])
fpr_log, tpr_log, _ = roc_curve(y_test, log_model.predict_proba(X_test)[:,1])

roc_auc_rf = auc(fpr_rf, tpr_rf)
roc_auc_log = auc(fpr_log, tpr_log)

plt.figure(figsize=(8, 6))
plt.plot(fpr_rf, tpr_rf, color='blue', label=f'Random Forest (AUC = {roc_auc_rf:.2f})')
plt.plot(fpr_log, tpr_log, color='green', label=f'Logistic Regression (AUC = {roc_auc_log:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.show()

# 🌟 Feature Importance (Random Forest)
feature_importances = pd.Series(rf_model.feature_importances_, index=X.columns)
feature_importances.sort_values(ascending=False).plot(kind='bar', figsize=(10, 6))
plt.title('Feature Importance - Random Forest')
plt.ylabel('Importance Score')
plt.show()

```

